

**DESENVOLVIMENTO DE SISTEMAS DE IA**

Turma UN-0226 — Asa Norte — Matutino

**SISTEMA DE RECOMENDAÇÃO HÍBRIDO PARA  
PLATAFORMA DE ENSINO SUPERIOR**

Manual de Execução e Consulta da API REST

Aluno

José Carvalho do Nascimento Junior

Professor

Fabio Oliveira Guimarães

Maio de 2026

## 1 INTRODUÇÃO

Este manual apresenta as instruções de execução do Sistema de Recomendação Híbrido desenvolvido para a disciplina Desenvolvimento de Sistemas de IA, Turma UN-0226 (Asa Norte, Matutino), ministrada pelo professor Fabio Oliveira Guimarães. O sistema é distribuído como um conjunto Docker composto por dois serviços: um treinador, responsável pelo pipeline offline de carga, ajuste dos modelos e cálculo das métricas de avaliação, e uma interface REST que serve as recomendações sob demanda. As próximas seções descrevem os pré-requisitos, o procedimento de inicialização e dois exemplos de consulta para cada endpoint disponível.

Os modelos integrados ao sistema são quatro: filtragem baseada em conteúdo, com TF-IDF aplicado à descrição dos materiais e atributos categóricos do aluno; filtragem colaborativa, via SVD truncada sobre a matriz aluno por material; recomendador baseado em conhecimento, com regras sobre áreas de interesse e nível esperado por período acadêmico; e um combinador híbrido, que normaliza e pondera os três sinais anteriores. A API REST permite acessar qualquer um dos quatro por parâmetro de consulta, viabilizando a comparação direta para um mesmo perfil de aluno.

## 2 PRÉ-REQUISITOS

A execução requer Docker Desktop instalado e em funcionamento. O ícone da baleia, na barra superior do macOS, deve estar ativo e estável antes do primeiro comando. Caso ainda não esteja instalado, o aplicativo está disponível em [docker.com/products/docker-desktop](https://docker.com/products/docker-desktop).

É necessário também que o caminho do projeto no qual o Compose será executado não contenha caracteres acentuados, em razão de uma limitação do BuildKit do Docker Desktop em algumas versões. Por esse motivo, o procedimento descrito a seguir copia a pasta original para `~/Documents/reco_hibrido` antes de iniciar o build. O arquivo `.env`, criado automaticamente, mantém o caminho do diretório local que contém os três arquivos CSV do dataset (`dados_alunos.csv`, `materiais_didaticos.csv` e `interacoes.csv`).

## 3 INICIALIZAÇÃO DO DOCKER

Após verificar que o Docker Desktop está aberto, abra o aplicativo Terminal do macOS e execute as três instruções abaixo, em sequência. A primeira copia o projeto para um caminho sem acentos. A segunda entra no diretório copiado. A terceira constrói a imagem, executa o pipeline de treino e inicializa a API.

```
cp -R "/Users/josecarvalho/Documents/Claude/Projects/Sistema de \
Recomendação Híbrido para Plataforma de Ensino" \
```

```
~/Documents/reco_hibrido
cd ~/Documents/reco_hibrido
docker compose up --build
```

A primeira execução leva entre dois e três minutos para construir a imagem, em função do download das dependências Python (FastAPI, scikit-learn, pandas e bibliotecas auxiliares). As execuções subsequentes reaproveitam a imagem em cache e levam cerca de trinta segundos para o pipeline de treino completo. Para encerrar, pressione Ctrl+C no Terminal no qual o Compose está em execução. Para reiniciar apenas a API, sem retreinar os modelos, basta executar `docker compose up api`.

## 4 ENDEREÇO LOCAL DA API

Com o Compose ativo, a API REST fica disponível na máquina local no endereço `http://localhost:8000`. Esse endereço é o ponto de entrada para todos os endpoints documentados na seção seguinte. A documentação interativa, gerada automaticamente pelo FastAPI, está em `http://localhost:8000/docs` e oferece um formulário para teste direto de cada endpoint, com autocomplete de parâmetros e exibição formatada da resposta. A especificação OpenAPI bruta está em `http://localhost:8000/openapi.json`.

## 5 ENDPOINTS E EXEMPLOS DE CONSULTA

Esta seção apresenta a relação dos sete endpoints da API e dois exemplos de consulta para cada um. Os exemplos utilizam o utilitário `curl`, disponível no Terminal do macOS, mas as mesmas URLs podem ser acessadas diretamente em qualquer navegador. Para visualização formatada do JSON, as chamadas via `curl` podem ser canalizadas para `python3 -m json.tool`.

### 5.1 Verificação de saúde — GET /health

Verifica se a aplicação está em execução e se os artefatos de treino foram carregados em memória. É o primeiro endpoint a ser consultado após a subida do Compose, para confirmar que o sistema está pronto.

```
Exemplo 1 – chamada simples
curl http://localhost:8000/health
```

```
Exemplo 2 – formatação legível
curl -s http://localhost:8000/health | python3 -m json.tool
```

### 5.2 Configuração do sistema — GET /info

Retorna o número de alunos e materiais indexados pelo modelo, o valor padrão de K, os pesos do combinador híbrido e a relação das estratégias disponíveis.

Exemplo 1

```
curl http://localhost:8000/info
```

Exemplo 2

```
curl -s http://localhost:8000/info | python3 -m json.tool
```

### 5.3 Métricas de avaliação — GET /metrics

Retorna o conteúdo do arquivo metrics.json gerado pelo treinador, com configuração utilizada, dados de integridade referencial, tamanho do subset e métricas Precision@K, Recall@K, F1, RMSE, Coverage e Diversity para os quatro modelos.

Exemplo 1 – todas as métricas

```
curl http://localhost:8000/metrics
```

Exemplo 2 – apenas as métricas do modelo híbrido

```
curl -s http://localhost:8000/metrics | \
python3 -c "import sys,json; d=json.load(sys.stdin); print([m for m in d['models'] if
```

### 5.4 Perfil do aluno — GET /alunos/{id\_aluno}

Retorna o cadastro do aluno, incluindo nome, curso, período acadêmico, lista de disciplinas cursadas e lista de áreas de interesse. Os identificadores válidos são inteiros entre 1 e 10000.

Exemplo 1 – aluno 1 (João Silva, Engenharia de Software, 5º período)

```
curl http://localhost:8000/alunos/1
```

Exemplo 2 – aluno 100

```
curl http://localhost:8000/alunos/100
```

### 5.5 Histórico de interações do aluno — GET /alunos/{id\_aluno}/historico

Lista os materiais que o aluno já consumiu no conjunto de treino, com avaliação média e contagem de interações por material. O parâmetro limit controla o número máximo de itens retornados (entre 1 e 200, padrão 20).

Exemplo 1 – cinco itens mais recentes do aluno 1

```
curl "http://localhost:8000/alunos/1/historico?limit=5"
```

Exemplo 2 – histórico ampliado do aluno 42

```
curl "http://localhost:8000/alunos/42/historico?limit=50"
```

### 5.6 Detalhes do material — GET /materiais/{id\_material}

Retorna os atributos do material: título, tipo (livro, artigo ou vídeo), área, nível de dificuldade, descrição textual e autor. Os identificadores válidos são inteiros entre 1 e 10000.

Exemplo 1 – material 1

```
curl http://localhost:8000/materiais/1
```

Exemplo 2 – material 100, formatado

```
curl -s http://localhost:8000/materiais/100 | python3 -m json.tool
```

### 5.7 Recomendações para o aluno — GET /recomendacoes/{id\_aluno}

Retorna os top-K materiais recomendados para o aluno informado, segundo a estratégia escolhida. Os parâmetros aceitos são k (entre 1 e 100, padrão 10), strategy (hybrid, content, collab ou knowledge, padrão hybrid) e excluir\_consumidos (true ou false, padrão true). O parâmetro excluir\_consumidos remove do ranking os materiais já presentes no histórico de treino, o que evita recomendar itens já vistos pelo aluno.

Exemplo 1 – cinco recomendações híbridas para o aluno 1  
`curl "http://localhost:8000/recomendacoes/1?k=5&strategy=hybrid"`

Exemplo 2 – dez recomendações colaborativas para o aluno 42,  
sem excluir os materiais já consumidos  
`curl "http://localhost:8000/recomendacoes/42?k=10\&strategy=collab&excluir_consumidos=false"`

## 6 OBSERVAÇÕES FINAIS

A interrupção do Compose com Ctrl+C preserva os artefatos no diretório ./artifacts/, de modo que a próxima inicialização com docker compose up api reutiliza os modelos já treinados sem repetir o pipeline. Para forçar um novo treino, com eventual ajuste dos hiperparâmetros via variáveis de ambiente no arquivo docker-compose.yml, executa-se docker compose run --rm trainer.

A documentação completa do projeto, incluindo a arquitetura interna do pipeline, a justificativa metodológica do tratamento de integridade referencial entre os três datasets e a discussão dos resultados das métricas comparativas, está consolidada no arquivo README.md, disponibilizado na raiz do repositório.